GIT INTERVIEW PREPARATION

Beginner to Advanced • Product-Based Companies • Up to 5-Year Experience

Simple-English answers, command examples, scenario-based questions, and a quick revision guide.

Purpose: a practical document to read anywhere and revise before an interview.

How to use this guide

- First reading: understand the concept and the short answer.
- Second reading: practice saying each answer aloud in your own words.
- Practice session: run the example commands in a temporary repository, never in an important project when learning destructive commands.
- Before the interview: review the last-minute checklist and the scenario answers.

Honesty rule: do not claim incidents, responsibilities, or five years of experience you have not actually had. If you have not faced a scenario, explain how you would approach it.

Contents

- 1. Git fundamentals
- 2. Everyday commands and flags
- 3. Branching and collaboration
- 4. Merge, rebase, cherry-pick, and conflicts
- 5. Undo, recovery, and safety
- 6. Remotes, history, tags, and advanced tools
- 7. Team workflow and product-company practices
- 8. Incident/scenario-based questions
- 9. Three-to-five-year experience questions
- 10. Embedded C / firmware scenarios
- 11. What-do-you-understand-by questions
- 12. Quick revision tables and final checklist

1. Git fundamentals

Q1. What is Git, and why do we use it?

Answer: Git is a distributed version control system. It tracks changes in source code, helps developers work independently on features, and lets teams review and recover earlier versions.

Q2. What is the difference between Git and GitHub?

Answer: Git is the version-control tool. GitHub is a hosting and collaboration platform for Git repositories, with features such as Pull Requests, reviews, permissions, and automated checks. Git also works with other hosting services or locally.

Q3. Explain the working directory, staging area, and repository.

Answer: The working directory contains the files I edit. The staging area contains changes selected for the next commit. The repository stores commits and history. I use git add to stage and git commit to record staged changes.

Q4. What is a commit?

Answer: A commit is a recorded snapshot of staged changes, with metadata such as author, time, and message. Meaningful commits create checkpoints that are easier to review, compare, and recover.

Q5. What is a branch?

Answer: A branch is a movable reference to a commit. It lets me work on a feature or fix independently from the main development line.

Q6. What are HEAD, main, origin, and upstream?

Answer: HEAD identifies the current checkout, usually the current branch or commit. main is a common branch name. origin is a common remote nickname. Upstream can mean a branch's tracking branch or, in fork workflows, a remote named upstream. These names are not all Git commands.

Q7. What is the difference between git init and git clone?

Answer: git init creates a new local repository in a directory. git clone copies an existing repository and its history to a local directory, and normally configures a remote.

Q8. What is the difference between git add and git commit?

Answer: git add selects changes for the next commit. git commit records the staged snapshot. I inspect staged changes with git diff --staged before committing.

Q9. What is the difference between git status, git diff, and git log?

Answer: git status reports branch and file state. git diff shows differences between versions or areas, such as working-tree and staged changes. git log shows commit history.

Q10. What is a distributed version control system?

Answer: Each clone normally contains repository history locally, so developers can inspect history and create commits without contacting the server. A remote is used to exchange changes with others.

2. Everyday commands and flags

Command	Purpose / interview explanation
git status	Shows the current branch, staged changes, unstaged changes, and untracked files.
git add <file>	Stages selected file changes for the next commit.
git add -p	Interactively selects parts of changes to stage.
git commit -m "message"	Creates a commit with the supplied message from staged changes.
git log --oneline --graph --decorate --all	Shows compact commit history and branch relationships.
git diff	Shows unstaged working-tree differences.
git diff --staged	Shows changes selected for the next commit.
git show <commit>	Shows details and changes for a commit or other Git object.
git switch <branch>	Switches to an existing branch.
git switch -c <branch>	Creates and switches to a new branch.
git branch -d <branch>	Deletes a local branch when Git considers it safely merged.
git remote -v	Shows configured remote names and fetch/push URLs.
git fetch origin	Downloads remote updates and updates remote-tracking references.
git pull	Fetches and then integrates changes, typically by merge or rebase according to configuration.
git push	Sends local commits to a remote when the branch and permissions allow.
git push -u origin <branch>	Pushes a new branch and sets upstream tracking.
git stash push -m 'message'	Temporarily shelves changes; check status and stash contents.
git tag -a v1.0 -m 'Release 1.0'	Creates an annotated tag, commonly used to mark a release.
git grep 'pattern'	Searches tracked files for matching text.
git bisect	Uses good/bad test results to narrow down which commit introduced a regression.
git worktree add <path> <branch>	Creates another working directory linked to the same repository.
git help <command>	Opens help for a command; use it to verify options for the installed version.

What do flags and arguments mean?

A command can contain the program, command, options/flags, and values. Example: git commit -m "Fix motor timeout". Here, git is the program, commit is the command, -m is an option, and the quoted text is its value.

Option / word	Meaning in context	Example
-m	Supplies a message for commands that	git commit -m "Fix bug"

	support it, such as commit.	
-b	With git checkout -b, creates and switches to a branch. It is not a universal flag.	git checkout -b feature/test
-c	With git switch -c, creates and switches to a branch.	git switch -c feature/test
-u	With git push, sets upstream tracking; long form is --set-upstream.	git push -u origin feature/test
--global	With git config, applies a setting for the current user across repositories.	git config --global user.name "Your Name"
--staged	With supported commands, refers to staged changes.	git diff --staged
--amend	With git commit, replaces the latest commit with a new one.	git commit --amend
--soft	With git reset, moves the branch reference but keeps changes staged.	git reset --soft HEAD~1
--mixed	With git reset, moves the reference and resets the index; working edits generally remain.	git reset HEAD~1
--hard	With git reset, resets the reference, index, and tracked working files; can discard work.	git reset --hard <commit>
--force-with-lease	Force-push with a check that the remote reference has not changed unexpectedly.	git push --force-with-lease
origin	A conventional remote name, not a flag or command.	git fetch origin
main	A common branch name, not a reserved command.	git switch main
HEAD	Reference to the current checkout.	git show HEAD
--	Separates options from path names in supported commands.	git restore -- motor.c

There is no single fixed number of Git flags. Each command has its own options, and capitalization can matter. Verify with git help <command> or git <command> -h.

3. Branching and collaboration

Q11. Describe your normal workflow for a new feature.

Answer: I check the working tree, update the base branch according to team policy, create a feature branch, implement and test the change, review the diff, stage only relevant files, commit, push, and open a Pull Request.

```
git status
 git switch main
 git pull --ff-only
 git switch -c feature/motor-stop
 git diff
 git add motor.c
 git diff --staged
 git commit -m "Add motor stop handling"
 git push -u origin feature/motor-stop
```

Q12. What is a Pull Request?

Answer: It is a hosting-platform collaboration feature that lets the team review, discuss, test, and approve changes before merging. It is not a Git command.

Q13. What is an upstream branch?

Answer: It is the remote-tracking branch associated with a local branch, helping Git know which remote branch to compare with and use for default pull/push behavior.

Q14. What is a fast-forward merge?

Answer: It moves the target branch pointer forward when the target has no separate commits to combine, so no merge commit is required.

Q15. What is a detached HEAD?

Answer: It means I checked out a specific commit instead of a branch. It is useful for inspecting old code. If I make commits I want to keep, I create a branch.

Q16. How do you delete a branch?

Answer: For a local branch, use git branch -d <branch> when safely merged. For a remote branch, use git push origin --delete <branch>. I check whether work is merged and whether the branch is still needed.

Q17. How do you compare branches?

Answer: I use git log to compare commits and git diff to compare file changes. For example, git log main..feature shows commits reachable from feature but not main. git diff main...feature compares the feature changes since the merge base.

Q18. What is the difference between fetch and pull?

Answer: Fetch downloads updates without integrating them into my current branch. Pull fetches and then integrates them, usually by merge or rebase. I often fetch first when I want to inspect incoming changes.

4. Merge, rebase, cherry-pick, and conflicts

Q19. Merge vs rebase?

Answer: Merge combines histories and may create a merge commit. Rebase replays commits on a new base, changing the IDs of the replayed commits. Merge preserves the integration history; rebase can create a linear history. I follow team policy and avoid casually rebasing shared commits.

Q20. What is cherry-pick?

Answer: It applies the change from a selected commit onto the current branch, creating a new commit. It is useful when a specific fix must be added to a release branch without merging all feature work.

```
git switch release
 git cherry-pick <commit>
```

Q21. How do you resolve a merge conflict?

Answer: I run git status, inspect each conflicted file, decide the correct combined content, remove conflict markers, run tests, stage the resolved files, and complete the merge. For a rebase, I use rebase --continue instead of merge --continue.

```
git status
# edit and test files
 git add <file>
 git merge --continue
```

Q22. How do you cancel a merge or rebase?

Answer: If I decide not to continue, I can use git merge --abort during a merge or git rebase --abort during a rebase, when Git can return to the prior state. I check status before and after.

Q23. What is git stash?

Answer: It temporarily shelves changes so I can switch tasks without creating a normal commit. stash pop applies and removes an entry if successful; stash apply applies it while retaining the entry. I check the working tree and stash list.

Q24. What is the difference between git merge and git cherry-pick?

Answer: Merge integrates branch history as a whole. Cherry-pick applies a selected commit's change. I choose based on whether I need the whole branch or only a specific independent fix.

5. Undo, recovery, and safety

Command	What it does	Typical use / caution
git restore <file>	Restores working-tree content from a source, usually the index by default.	Can discard unstaged edits to that file.
git restore --staged <file>	Unstages a file while retaining working-tree edits.	Useful when a file was staged accidentally.
git reset --soft HEAD~1	Moves branch reference back one commit and keeps changes staged.	Usually for a local, unshared commit.
git reset HEAD~1	Moves reference and resets staging; working edits generally remain.	Understand the state before running it.
git reset --hard <commit>	Resets branch reference, index, and tracked files.	Can discard uncommitted tracked changes.
git revert <commit>	Creates a new commit that reverses an earlier commit.	Often preferred for undoing shared changes.
git reflog	Shows recent local reference movements.	Can help find a prior commit after reset/rebase; entries expire.

Q25. How are reset, revert, and restore different?

Answer: Restore works on file content or staging. Reset moves the branch reference and may update the index and working tree depending on mode. Revert creates a new commit that reverses an earlier commit. For shared history, revert is often safer.

Q26. You committed to the wrong branch. What do you do?

Answer: I inspect status and log first. If the commit is local, I can cherry-pick it to the correct branch and then carefully correct the original branch. If it is already shared, I coordinate before rewriting history.

```
git log --oneline -5
 git switch correct-branch
 git cherry-pick <commit>
```

Q27. You accidentally ran reset --hard. Can you recover?

Answer: I inspect reflog to see whether a previous commit can be recovered, then create a recovery branch at the correct commit. But uncommitted edits discarded by reset --hard may not be recoverable through Git.

```
git reflog
 git switch -c recovery-branch <commit>
```

Q28. You pushed a bad commit. What do you do?

Answer: I assess the impact and identify the faulty commit. If it is shared, I usually revert it, test the correction, and follow the team's review/release process. I do not rewrite shared history casually.

```
git revert <bad-commit>
```

Q29. You accidentally staged a secret. What do you do?

Answer: If it has not been committed, I unstage it and ensure it is not included. If it has been pushed, I treat it as exposed, revoke or rotate it, and follow the security process. Removing it later does not make it safe.

```
git restore --staged -- <file>
```

Q30. What is force-push and when is it dangerous?

Answer: Force-push rewrites a remote branch reference and can remove commits from the visible branch history. It may be needed after a deliberate rebase, but must be coordinated. --force-with-lease adds a check against unexpected remote updates but is not risk-free.

6. Remotes, history, tags, and advanced tools

Q31. How do you find which commit introduced a bug?

Answer: If I can reproduce the bug and know a good revision, I use git bisect to narrow the commit range by testing candidate commits. I finish with git bisect reset.

```
git bisect start
 git bisect bad
 git bisect good <known-good-commit>
# test each candidate, then mark good/bad
 git bisect reset
```

Q32. What is git worktree?

Answer: It lets me create another working directory linked to the same repository. This can help me work on a hotfix while keeping another branch checked out in the main folder.

Q33. What is a tag?

Answer: A tag is a named reference to a commit, often used to identify a release. An annotated tag stores extra metadata and a message. Published release tags should not be moved casually.

Q34. What are Git hooks and CI checks?

Answer: Hooks are scripts triggered by local Git events, such as before a commit. CI checks run automated builds/tests in a shared environment. Critical policies should not rely only on local hooks because they can often be bypassed.

Q35. What do .gitignore and .gitattributes do?

Answer: .gitignore lists patterns for files Git should normally ignore, especially generated output or local files. It does not automatically untrack already tracked files. .gitattributes defines repository-specific handling such as line endings and file types.

Q36. How do you investigate an unclear history?

Answer: I use git log --oneline --graph --decorate --all to inspect history, git show to inspect commits, git diff to compare changes, and reflog to find recent local reference movements. I gather evidence before changing history.

Q37. Should binaries be committed?

Answer: It depends on the project. Source and build configuration are normally tracked. Required release binaries may be stored according to team policy; large binaries may suit Git LFS or an artifact repository. I ensure artifacts can be traced to a source commit and build configuration.

7. Team workflow and product-company practices

Q38. How do you keep history clean?

Answer: I make small, meaningful commits, avoid unrelated changes, review staged diffs, and follow the team's merge/rebase policy. I may squash temporary commits before merging if the team prefers it. I avoid rewriting shared history without agreement.

Q39. A push is rejected because the remote has new commits. What do you do?

Answer: I fetch and inspect remote changes, integrate them using the team's approved merge/rebase approach, resolve conflicts, test, and push again. I do not force-push simply to bypass a rejection.

```
git fetch origin
 git status
 git log --oneline --graph --all
```

Q40. A branch is several weeks behind main. How do you update it?

Answer: I inspect the changes in main, fetch updates, and integrate them using the team's chosen merge or rebase approach. Then I resolve conflicts, run tests, and push. If rebasing a shared branch, I coordinate because commit IDs change.

Q41. What practices would you recommend for a team?

Answer: Use focused commits, clear branch names, Pull Request reviews, automated build/test checks, secret scanning, protected branches, documented merge/release policies, and traceable release tags. Agree on urgent-fix and artifact-handling procedures.

8. Incident/scenario-based questions

Q42. Two developers changed the same driver lines and a conflict occurs. What do you do?

Answer: I check status, compare both intended changes, combine the correct behavior rather than blindly choosing one side, build and test, then stage and continue the operation. I communicate with the other developer if the intended behavior is unclear.

Q43. A feature works on one machine but fails on another. How can Git help?

Answer: I compare commit IDs, tracked configuration, source changes, toolchain versions, and build instructions. Git helps identify code differences, but I also investigate environment and dependency differences because Git alone does not guarantee reproducible builds.

Q44. A release tag points to firmware with a critical bug. What do you do?

Answer: I confirm the tagged commit and impact, create a fix on the appropriate maintenance branch, test it, and release a new version with a new tag according to policy. I avoid silently moving a published tag because other systems may depend on it.

Q45. A force-push appears to have removed another developer's commits. What do you do?

Answer: I stop further history changes and inspect logs, reflogs, other clones, and remote references. After identifying the missing commits, I coordinate with the team to restore them safely. I do not assume they are permanently lost or force-push again immediately.

Q46. A commit contains an unrelated file or generated build output. What do you do?

Answer: Before pushing, I inspect git show --stat HEAD and the staged diff. If the commit is local, I can amend it or create a corrective commit depending on the situation. I update ignore rules where appropriate, remembering that ignore rules do not untrack existing files automatically.

9. Three-to-five-year experience questions

Q47. When would you choose rebase over merge?

Answer: I choose rebase to update a private feature branch and keep a linear history. I choose merge when preserving integration points or shared branch history is important. The team's policy and whether others depend on the commits guide the decision.

Q48. How do you manage a hotfix for a stable release?

Answer: I identify the release branch and required fix, make a focused change, review and test it, and cherry-pick only if the fix is appropriate and independent. I verify compatibility, obtain review, and tag the tested release according to policy.

Q49. How do you investigate a missing commit?

Answer: I check the normal log and all branches, then inspect reflog and other clones if needed. I identify the exact commit before creating a recovery branch or changing references. Reflog is local and temporary, so recovery is not guaranteed.

Q50. How would you prevent recurring Git integration problems?

Answer: I encourage smaller changes, frequent integration, automated tests, clear ownership, reviews, and a documented branching strategy. I look for root causes such as long-lived branches, missing tests, unclear file ownership, or inconsistent toolchains.

Q51. What do you do if a teammate asks you to force-push a shared branch?

Answer: I ask why history rewriting is necessary, check who may depend on the branch, and coordinate with the team. If approved, I verify the target and prefer --force-with-lease. I never treat it as a routine fix for every push problem.

Q52. How do you ensure a firmware release is traceable?

Answer: I identify the source commit, tag the tested release, record build/toolchain configuration and relevant hardware compatibility, and store release artifacts according to team policy. I verify that the delivered binary matches the intended build.

10. Embedded C / firmware scenarios

Q53. A motor-control fix is on development but needed in a release branch. What do you do?

Answer: I identify the exact tested commit, check the release branch, and cherry-pick the fix only if it is compatible and sufficiently independent. I resolve conflicts, build and test the firmware, and follow release review procedures.

Q54. How do you reduce risk when two developers edit a hardware driver?

Answer: I divide work where practical, use separate branches, keep commits focused, integrate regularly, review conflicts carefully, and run relevant builds and hardware tests. I never blindly select one side of a conflict.

Q55. Would you commit generated firmware binaries?

Answer: I follow the project's traceability and artifact policy. Source, configuration, linker scripts, and build instructions should be tracked. Release binaries may be stored as release artifacts or in Git LFS if appropriate, with links to the source commit and build configuration.

Q56. How do you handle secrets in firmware repositories?

Answer: I never commit tokens, private keys, passwords, or production credentials. If a secret is exposed, I rotate/revoke it immediately and follow the team's incident process. I also review ignore rules and secret-scanning checks.

Q57. How do you make firmware changes reviewable?

Answer: I keep commits focused, explain the behavior change, include relevant tests or build evidence, and avoid unrelated generated files. For hardware-dependent behavior, I record the test setup and result so reviewers can understand the validation.

11. What-do-you-understand-by questions

Term	Simple answer
Version control	Tracking changes over time so people can compare, collaborate, and recover versions.
Remote repository	Another repository used to exchange commits with a server or other developers.
Merge conflict	A situation where Git cannot automatically combine changes.
Clean working tree	No uncommitted changes are reported by status; ignored files may still exist.
Protected branch	A hosting-platform rule limiting actions such as direct pushes or deletion.
Release tag	A named reference to a specific commit used to identify a version.
Hotfix	A focused fix for an urgent issue, handled with controlled testing and release.
Code review	Reviewing changes for correctness, maintainability, and requirements before integration.
Upstream tracking	The remote-tracking branch associated with a local branch.
Staging area	The index where selected changes are prepared for the next commit.
Detached HEAD	A checkout at a commit rather than a branch.
Force-push	A push that updates a remote reference in a way that can rewrite its visible history.

12. Last-minute revision sheet

Most useful daily sequence

git status → git diff → git add <file> → git diff --staged → git commit -m "Clear message" → git push

New feature branch

git switch -c feature/your-change → implement and test → review diff → commit → git push -u origin feature/your-change

Before changing history

- Check git status and git log.
- Confirm whether the commit has been pushed or shared.
- Choose restore, reset, revert, or rebase based on what must change.
- Avoid reset --hard and force-push unless the consequences are understood and approved.

Last-minute interview checklist

- Can I explain Git vs GitHub?
- Can I explain working directory vs staging area vs commit?
- Can I compare fetch vs pull, merge vs rebase, reset vs revert vs restore?
- Can I resolve and abort a merge/rebase conflict?
- Can I explain origin, main, HEAD, upstream, and common flags?
- Can I explain how I would recover a missing commit without guessing?
- Can I describe a feature branch and Pull Request workflow?
- Can I explain how I would protect secrets and trace a firmware release?
- Can I give truthful examples from my own work and explain what I learned?

Final guidance

This guide is a practical interview preparation reference, not an exhaustive list of every Git command or option. Git options can vary by command and version; confirm exact syntax with git help <command>. For scenario questions, structure your answer as: understand the current state → choose the safest approach → execute → verify and test.

For 3–5-year roles, interviewers may expect deeper design and collaboration reasoning. Tailor the examples to your real responsibilities, and never claim an incident or experience you did not have.